

Phase 2 Plan: The NLI Cross-Encoder Decision Gate & RLAIIF Engine

Objective: Implement the second tier of the EAV-RAG architecture. While Phase 1 established the vector database, Phase 2 deploys a Natural Language Inference (NLI) Cross-Encoder decision gate to filter out non-entailing chunks and integrates GBNF grammar constraints with Qwen-2.5-1.5B to eliminate generation hallucinations.

Detailed Team Assignments & Execution Protocols

1. Core NLI Cross-Encoder Logic & Pipeline Integration (Assigned to M1: Khushi)

Role: Machine Learning Architect

Objective: Develop the sequence classification filter that sits between ChromaDB retrieval and the text generator, ensuring only verified factual evidence is passed forward.

- **Model Selection & Setup:** Integrate a lightweight transformer-based Cross-Encoder model optimized for NLI tasks (e.g., cross-encoder/nli-deberta-v3-small) inside the `src/cross_encoder_gate.py` script.
- **Pairwise Scoring Architecture:** Write the core evaluation logic. When ChromaDB returns the top-15 candidate chunks, the script must pair the user query (as the premise) with each individual chunk (as the hypothesis).
- **Threshold Classification:** Implement a scoring function that outputs multi-class probabilities (Entailment, Neutral, Contradiction). Define a strict filtering threshold (e.g., dropping any chunk where the *Entailment* probability falls below 0.85).
- **Pipeline Orchestration:** Connect the filtered output directly to the generation interface, ensuring only verified chunks form the context window.

2. Automated RLAIIF Dataset Generation Script (Assigned to M4: Praisyy)

Role: Data Generation & Preprocessing Lead

Objective: Automate the creation of a labeled training dataset (Query, Chunk, Label triplets) using Reinforcement Learning from AI Feedback (RLAIIF) principles to fine-tune or evaluate the Cross-Encoder gate.

- **Script Development (`src/generate_rlaif_data.py`):** Author a script that loads the Phase 1 ChromaDB index and a predefined list of 50 complex aquaculture evaluation queries.
- **Automated Pairing & Evaluation Prompting:** Program the script to fetch the top-5 retrieved chunks per query and pass them through a structured teacher evaluation prompt (utilizing an API or local evaluator model) to judge whether the chunk explicitly answers the query.
- **Dataset Export:** Automatically format the evaluated pairs into standard classification labels (1 for direct entailment, 0 for neutral/contradiction).
- **Deliverable:** Export the final synthesized training set to `data/processed/rlaif_train_dataset.csv` and commit it to the repository.

3. SLM Integration & GBNF Grammar Constraints (Assigned to M2: Meghana)

Role: Integration & Generation Lead

Objective: Program the local generation module using Qwen-2.5-1.5B and enforce structural determinism via GBNF grammar rules.

- **Model Loading (`src/generator.py`):** Set up the local execution environment for Qwen-2.5-1.5B (using llama.cpp bindings or transformers with 4-bit quantization) to ensure it runs completely offline.
- **Grammar Definition (`constraints.gbnf`):** Author a formal GBNF (GGML BNF) grammar file that enforces a strict JSON output schema. The schema must require specific keys:

JSON

```
{
  "diagnosis": "string",
  "recommended_action": "string",
  "confidence_score": "float",
  "supporting_evidence_id": "string"
}
```

- **Constraint Enforcement:** Integrate the grammar constraints into the generation call. Test the model to guarantee it cannot output conversational filler, markdown prose, or unformatted text, locking its responses entirely into the required JSON template.

4. Documentation, Literature & Visual Architecture (Assigned to M3: Jyothish)

Role: Research Lead & Project Manager

Objective: Maintain academic rigor, log engineering parameters, and construct the Phase 2 system flowcharts for the IEEE manuscript.

- **Literature Extension (Zotero):** Add foundational references regarding NLI Cross-Encoders, RLAIIF methodologies, and constrained decoding (GBNF) to the shared Zotero library, ensuring proper IEEE formatting.
- **Engineering Logboard Updates:** Document the technical specifications in the shared logbook:
 - *Cross-Encoder Model:* cross-encoder/nli-deberta-v3-small selected for balance between classification accuracy and edge inference latency.
 - *Entailment Threshold:* Set at ≥ 0.85 to strictly prevent neutral or contradictory chunks from entering the SLM context window.
 - *Generation Constraints:* GBNF grammar implementation details for deterministic JSON compliance.
- **Phase 2 Architecture Diagram:** Design a vector flowchart in Draw.io depicting the complete end-to-end pipeline:
User Query -> BGE-M3 Retrieval (Top-15) -> NLI Cross-Encoder Gate -> {Filtered Entailing Chunks -> GBNF-Constrained Qwen-2.5-1.5B -> {Structured JSON Output

Export the diagram at 300 DPI (docs/figures/phase2_architecture.png).

Phase 2 Step-by-Step Execution Guide: Methodological Approach for Each Team Member

M1: Khushi — Core NLI Cross-Encoder Logic (cross_encoder_gate.py)

Step-by-Step Implementation Approach:

1. **Environment Setup:** Ensure the cross-encoder dependency is installed within the virtual environment by running `pip install sentence-transformers`.
2. **Model Initialization:** Write a Python script that loads the sequence classification model using the Hugging Face CrossEncoder class:

```
from sentence_transformers import CrossEncoder
model = CrossEncoder("cross-encoder/nli-deberta-v3-small")
```
3. **Pairwise Construction Logic:** When ChromaDB returns the top-15 candidate text chunks for a given query, construct an array of input pairs formatted precisely as `[query, chunk_text]`.
4. **Scoring and Inference:** Pass the array into the model using `model.predict(pairs)`. The model will output multi-class logits or probabilities mapped to three standard NLI indices: *Contradiction*, *Entailment*, and *Neutral*.
5. **Threshold Enforcement:** Apply a strict logical filter to retain only chunks where the *Entailment* probability score meets or exceeds \$0.85\$. Wrap this function cleanly so it can accept a query and a list of chunks, returning only the verified subset to pass downstream.

M4: Praisya — Automated RLAIIF Dataset Generation (generate_rlaif_data.py)

Step-by-Step Implementation Approach:

1. **Define the Evaluation Seed:** Create a Python list containing 50 diverse aquaculture evaluation queries covering water quality parameters, disease symptoms, and feeding rates.
2. **Retrieve Candidate Contexts:** Write a script that loads the Phase 1 ChromaDB index (`data/chroma_db/`) and queries it using each seed question to extract the top-5 corresponding text chunks.
3. **Automated Teacher Labeling:** Program a structured evaluation loop. For each (Query, Chunk) pair, prompt a local evaluation rubric or teacher function to verify if the chunk contains the exact answer:
 - If yes, assign label 1 (Entailment).
 - If no, assign label 0 (Neutral/Contradiction).
4. **Format and Export:** Structure the results into a pandas DataFrame containing columns: `query_id`, `query`, `chunk_id`, `chunk_text`, and `label`. Save the final file to `data/processed/rlaif_train_dataset.csv` and push it to the repository.

M2: Meghana — SLM Integration & GBNF Grammar Constraints (generator.py & constraints.gbnf)

Step-by-Step Implementation Approach:

1. **Environment Setup:** Install local inference bindings by running `pip install llama-cpp-python` to execute Qwen-2.5-1.5B offline.
2. **Author the Grammar File (constraints.gbnf):** Create a formal GBNF text file in the root or `src/` directory enforcing rigid JSON structure:

```
root ::= object
object ::= "{" ws "\""diagnosis\""" ws ":" ws string "," ws "\""recommended_action\""" ws ":" ws string "," ws "\""confidence_score\""" ws ":" ws number "," ws "\""supporting_evidence_id\""" ws ":" ws string "}"
```

string ::= "\"" ([^"\\] | "\\" .) * "\""

number ::= [0-9]+ ("." [0-9]+)?

ws ::= [\t\n]*

3. **Model Loading & Constraint Execution:** Write src/generator.py to load the quantized Qwen model via llama_cpp.Llama with the grammar_path="constraints.gbnf" argument enabled.
4. **Validation Testing:** Test the generator script using filtered chunks from the Cross-Encoder gate to confirm that outputs strictly adhere to the defined JSON schema without conversational preamble.

M3: Jyothish — Documentation, Literature & Visual Architecture

Step-by-Step Implementation Approach:

1. **Zotero Extension:** Search for seminal papers and documentation regarding Natural Language Inference models, DeBERTa-v3 cross-encoders, and constrained decoding (GBNF grammar). Add them to the shared Zotero library under the EAV-RAG Phase 2 collection and format citations as IEEE.
2. **Logbook Documentation:** Open the shared project logbook and log the architectural parameters for Phase 2:
 - *Model Specification:* cross-encoder/nli-deberta-v3-small
 - *Decision Threshold:* Entailment probability ≥ 0.85
 - *Grammar Rule:* Strict JSON enforcement via GGML BNF syntax.
3. **Diagram Generation:** Open Draw.io and create a clean, professional vector diagram representing the Phase 2 pipeline. Ensure the visual sequence clearly steps from User Query -> Bi-Encoder Retrieval -> NLI Cross-Encoder Gate -> GBNF-Constrained Qwen Generation.
4. Export the final asset as a 300 DPI PNG (docs/figures/phase2_architecture.png).